Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт

по лабораторной работе №3
по курсу «Технологии глубоких нейронных сетей»
на тему «Основы Pytorch»

Выполнил:
студент группы 23ВВИ2
Юров Д.М.

Принял:
Митрохин М.А.

Пенза 2026

Цель работы: познакомиться с библиотекой PyTorch – изучить основные операции над тензорами и механизмом автоматического дифференцирования, а также применить Pytorch для решения задачи классификации.

Задание:

1. Установка PyTorch
   1) Откройте Anaconda Prompt или консоль команд, активируйте созданную в первой лабораторной работе среду (conda activate dnnenv);
   2) В браузере откройте страницу «https://pytorch.org/»;
   3) Прокрутите страницу вниз до раздела «Install pytorch»;
   4) В разделе установки выберите:
      - версию сборки pytorch (лучше оставить версию Stable);
      - операционную систему, установленную на Вашем компьютере;
      - пакетный менеджер для установки (в Anaconda Prompt можно использовать как conda, так и pip);
      - если Ваша видеокарта поддерживает технологию CUDA выше версии 11.8 (видеокарта производства Nvidia моделей старше поколения 10xx) следует выбрать установку с поддержкой CUDA, в противном случае лучше выбрать CPU.
   5) В результате будет сформирована команда для установки, которую необходимо скопировать в консоль команд и выполнить;
   6) После установки в консоли запустите python и выполните команды import torch и torch.__version__, в результате в консоль выведется версия установленного пакета PyTorch;
   7) Закройте интерпретатор python командой exit() и запустите IDE Spyder.
2. Знакомство с PyTorch
   1) Откройте файл «lab3_pytorch_basics.py»;

2) Прочитайте комментарии и выполните код;
3) После ознакомления с кодом выполните Задание 1.

3. Задание 1:
   1) Создайте тензор x целочисленного типа, хранящий случайное значение;
   2) Преобразуйте тензор к типу float32;
   3) Проведите с тензором x ряд операций:
      - возведение в степень n, где n = 3, если ваш номер по списку группы в ЭИОС – четный и n = 2, если ваш номер по списку группы в ЭИОС – нечетный;
      - умножение на случайное значение в диапазоне от 1 до 10;
      - взятие экспоненты от полученного числа.
   4) Вычислите и выведите на экран значение производной для полученного в п.3 значения по x.
4. Задание 2:
   1) На основе кода обучения линейного алгоритма создать код для решения задачи классификации цветков ириса из лабораторной работы №2.

Ход выполнения лабораторной работы:

1. Установка PyTorch.

1) Открыл Anaconda Prompt, активировал созданную в первой лабораторной работе среду «dnnenv» (рис. 1).

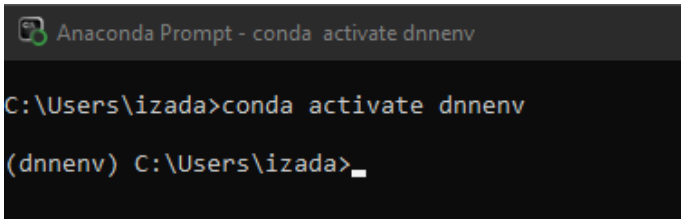


Рисунок 1 - активация среды "dnnenv"

2) В браузере открыл страницу «https://pytorch.org/» и пролистал ниже, до раздела «Install PyTorch». В разделе установки выбрал подходящие параметры и скопировал команду для установки (рис. 2).

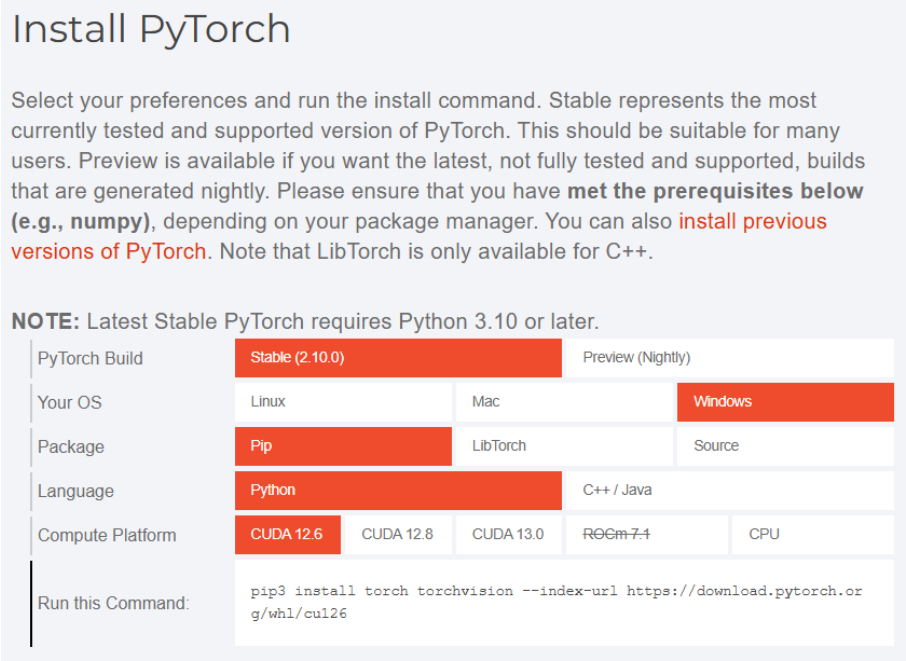


Рисунок 2 - раздел "Install PyTorch"

3) Вставил команду установки в командную строку «Anaconda Prompt» (рис. 3). После чего установились все необходимые пакеты.

```
(base) C:\Users\izada>conda activate dnnenv

(dnnenv) C:\Users\izada>pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126
```

Рисунок 3 - установка пакетов с помощью команды

4) Запустил интерпретатор Python и проверил текущую версию PyTorch (рис. 4).

```
(dnnenv) C:\Users\izada>python
Python 3.10.20 | packaged by Anaconda, Inc. |
Type "help", "copyright", "credits" or "licen
>>> import torch
>>> torch.__version__
'2.10.0+cu126'
```

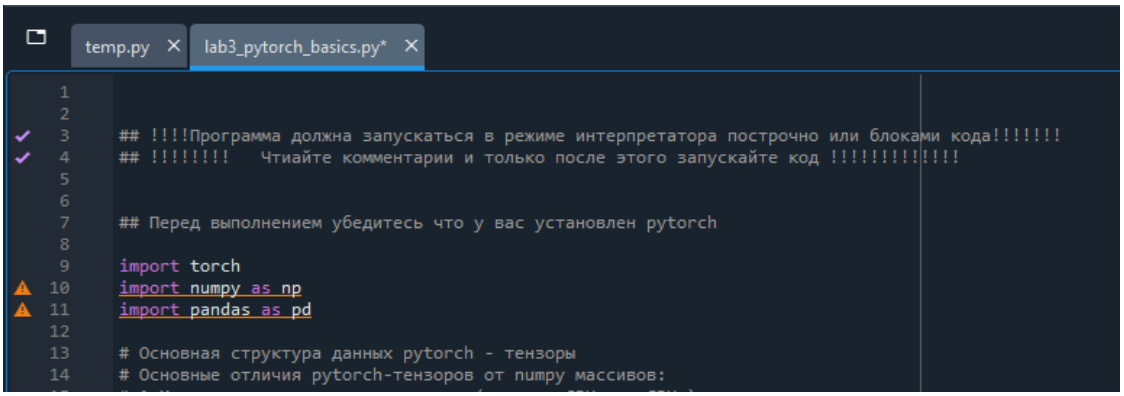
Рисунок 4 – проверка версии PyTorch

5) Из «Anaconda Prompt» запустил «Spyder» (рис. 5)

Рисунок 5 - запуск "Spyder"

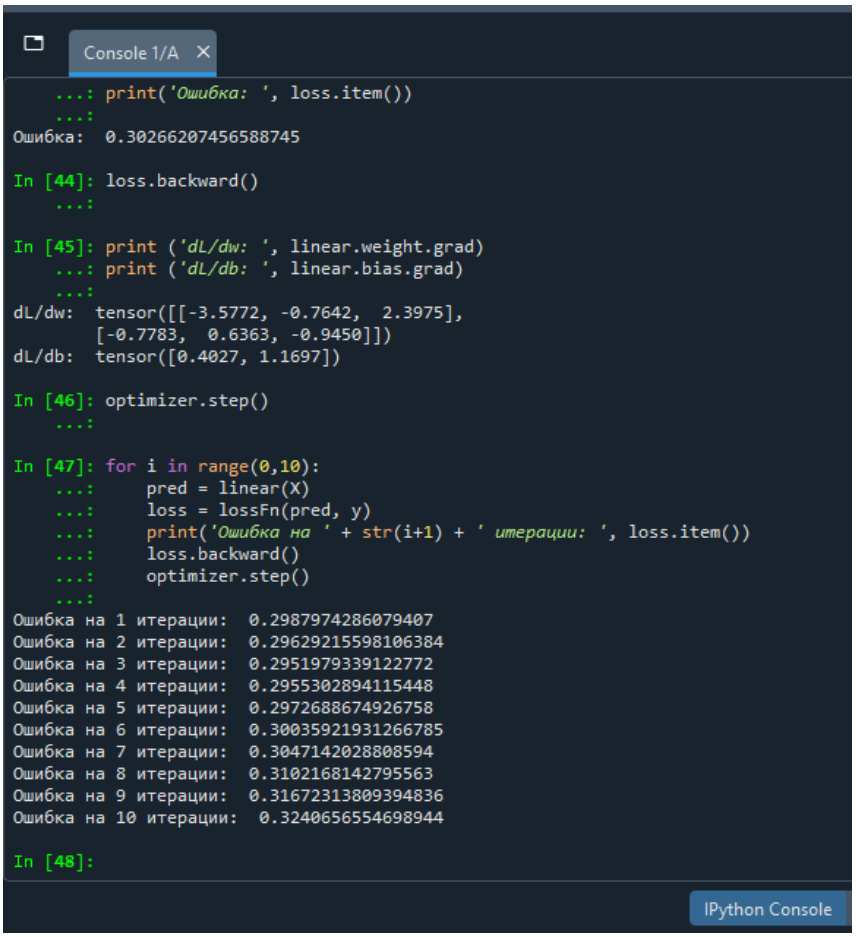
2. Знакомство с PyTorch

1) Открыл файл «lab3_pytorch_basics.py» (рис. 6), прочитал комментарии и выполнил код (рис. 7).



```
temp.py   lab3_pytorch_basics.py*

## !!!!Программа должна запускаться в режиме интерпретатора построчно или блоками кода!!!!!!!
## !!!!!!!!   Чтиайте комментарии и только после этого запускайте код !!!!!!!!!!!!!!


## Перед выполнением убедитесь что у вас установлен pytorch

import torch
import numpy as np
import pandas as pd

# Основная структура данных pytorch - тензоры
# Основные отличия pytorch-тензоров от numpy массивов:
```

Рисунок 6 - файл "lab3_pytorch_basics.py"



```
Console 1/A

    ...: print('Ошибка: ', loss.item())
    ...:
Ошибка:  0.30266207456588745

In [44]: loss.backward()
    ...:

In [45]: print ('dL/dw: ', linear.weight.grad)
    ...: print ('dL/db: ', linear.bias.grad)
    ...:
dL/dw:  tensor([[-3.5772, -0.7642,  2.3975],
        [-0.7783,  0.6363, -0.9450]])
dL/db:  tensor([0.4027, 1.1697])

In [46]: optimizer.step()
    ...:

In [47]: for i in range(0,10):
    ...:     pred = linear(X)
    ...:     loss = lossFn(pred, y)
    ...:     print('Ошибка на ' + str(i+1) + ' итерации: ', loss.item())
    ...:     loss.backward()
    ...:     optimizer.step()
    ...:
Ошибка на 1 итерации:  0.2987974286079407
Ошибка на 2 итерации:  0.29629215598106384
Ошибка на 3 итерации:  0.2951979339122772
Ошибка на 4 итерации:  0.2955302894115448
Ошибка на 5 итерации:  0.2972688674926758
Ошибка на 6 итерации:  0.30035921931266785
Ошибка на 7 итерации:  0.3047142028808594
Ошибка на 8 итерации:  0.3102168142795563
Ошибка на 9 итерации:  0.31672313809394836
Ошибка на 10 итерации:  0.3240656554698944

In [48]:

IPython Console
```

Рисунок 7 - результат выполнения кода

3. Задание 1

1) Написал код по заданию 1 (рис. 8) и выполнил его (рис. 9).

```python
import torch
import numpy as np
##import pandas as pd

import torch.nn as nn

import random

if(torch.cuda.is_available()):
    device = "cuda:0"
else:
    device = "cpu"

device

z = np.random.randint(0, 10, size=(1), dtype=np.int32)

x = torch.from_numpy(z).to(device)
print(x)

x = x.to(dtype=torch.float32)
x.requires_grad=True
print(x)

id_in_group = 26

if(id_in_group % 2 == 0):
    n = 3;
else:
    n = 2;
print("n = ", n, "\n")

y = x ** n
print(y)

y = y * random.randint(1, 10)
print(y)


y = torch.exp(y)
print(y)

y.backward()

print(x.grad)
print(y.grad_fn)
```

Рисунок 8 - код задания 1

```
tensor([2], device='cuda:0', dtype=torch.int32)
tensor([2.], device='cuda:0', requires_grad=True)
n =  3

tensor([8.], device='cuda:0', grad_fn=<PowBackward0>)
tensor([32.], device='cuda:0', grad_fn=<MulBackward0>)
tensor([7.8963e+13], device='cuda:0', grad_fn=<ExpBackward0>)
tensor([3.7902e+15], device='cuda:0')
<ExpBackward0 object at 0x0000023BBEC36290>
```

Рисунок 9 - результат выполнения кода задания 1

4. Задание 2

1) На основе кода обучения линейного алгоритма создал код для решения задачи классификации цветков ириса из лабораторной работы №2 (рис. 10).

```python
import torch
import torch.nn as nn
import pandas as pd
import numpy as np

if(torch.cuda.is_available()):
    device = "cuda:0"
else:
    device = "cpu"

print("device = ", device, "\n")

filename = 'iris.csv'
df = pd.read_csv(filename)

print("samples:\n", df.head(10))

x = df.iloc[:, [0,1,2,3]].values
y = df.iloc[:, 4].values

y = np.where(y == "Setosa", 0,
    np.where(y == "Versicolor", 1, 2))

x = torch.tensor(x, dtype=torch.float32).to(device)
y = torch.tensor(y, dtype=torch.float32).to(device)

y_new = torch.zeros((y.shape[0], 3), dtype = torch.float32).to(device)

for i in range(y.shape[0]):
    if y[i] == 0:
        y_new[i][0] = 1
    elif y[i] == 1:
        y_new[i][1] = 1
    else:
        y_new[i][2] = 1

y = y_new

print(x.shape)
#print(x)

print(y.shape)
#print(y)


linear = nn.Linear(4, 3).to(device)

print ('w: ', linear.weight)
print ('b: ', linear.bias)

lossFn = nn.MSELoss() # MSE - среднеквадратичная ошибка, вычисляется как sqrt(sum(y^2 - yp^2))

optimizer = torch.optim.SGD(linear.parameters(), lr=0.01) # lr - скорость обучения

yp = linear(x)

# имея предсказание можно вычислить ошибку
#loss = lossFn(yp, y)
#print('Ошибка: ', loss.item())

# и сделать обратный проход, который вычислит градиенты (по ним скорректируем веса)
#loss.backward()

for i in range(0, 150):
    optimizer.zero_grad()

    pred = linear(x)
    loss = lossFn(pred, y)

    loss.backward()
    optimizer.step()

    if((i+1) % 10 == 0):
        print('Ошибка на ' + str(i+1) + ' итерации: ', loss.item())

with torch.no_grad():
    pred_class = torch.argmax(pred, dim=1)
    true_class = torch.argmax(y, dim=1)
    acc = (pred_class == true_class).float().mean()

print("предсказанные классы:\n", pred_class)
print("истинные классы:\n", true_class)
print("точность =", acc.item())
```

Рисунок 10 - код для решения задачи классификации

2) Выполнил код программы (рис. 11).

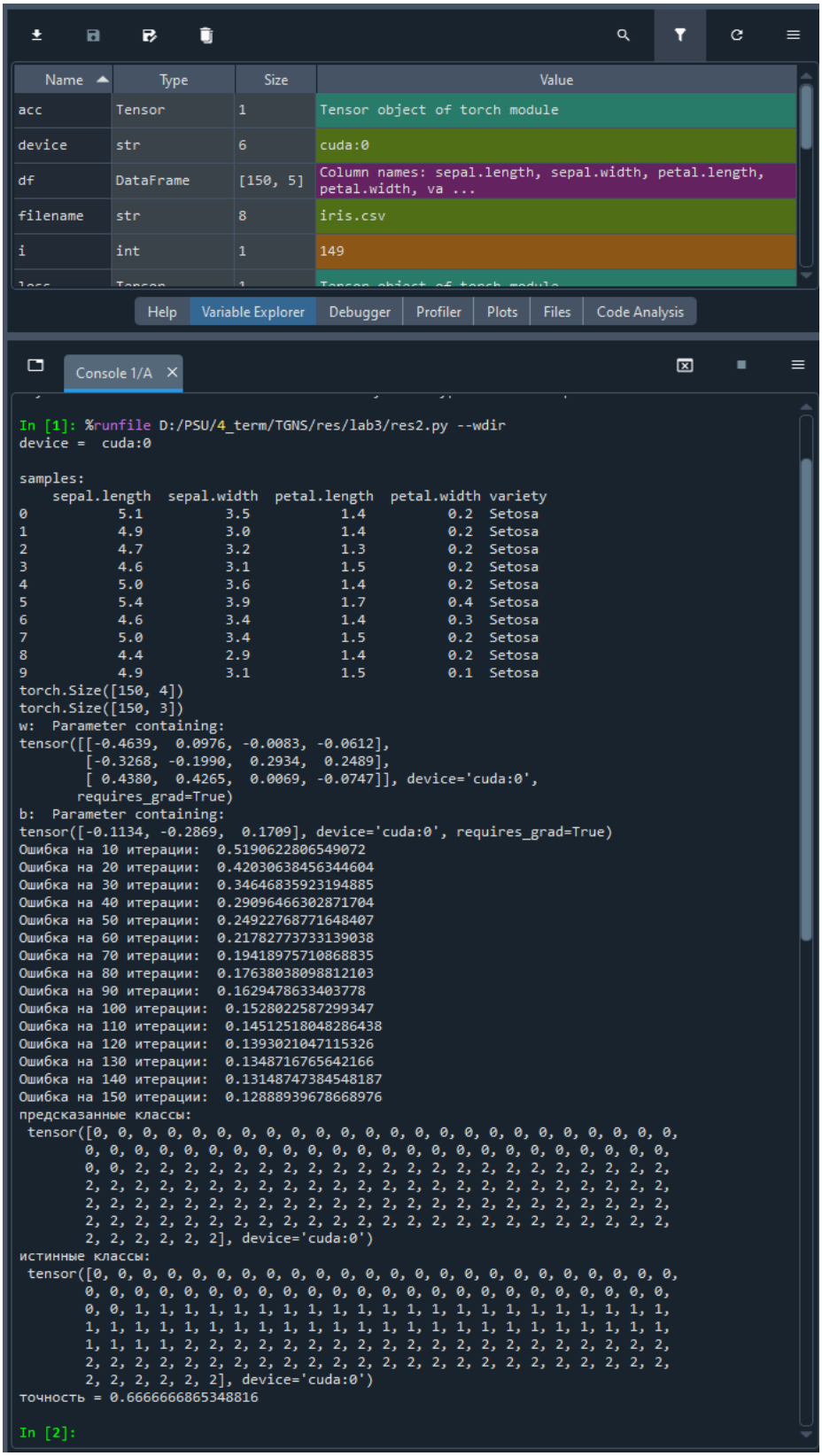


Рисунок 11 - результат выполнения программы

Вывод: в ходе выполнения лабораторной работы познакомился с библиотекой PyTorch – изучил основные операции над тензорами и механизмом автоматического дифференцирования, а также применил Pytorch для решения задачи классификации.